Experiment 2

Title

Apply various text preprocessing techniques for any given text: Tokenization, Filtration and Script Validation

Case Study: Resume Keyword Extraction

Theory

Text preprocessing is one of the fundamental steps in Natural Language Processing (NLP). Real world textual data is usually unstructured and contains unnecessary information such as punctuation marks, numbers, special symbols, and inconsistent formatting. Before applying machine learning or NLP techniques, it is important to clean and organize the text so that it becomes suitable for analysis.

In Resume Keyword Extraction, textual information is extracted from resumes to identify important skills, qualifications, technologies, and job-related keywords. The quality of the extracted text directly affects the performance of keyword extraction systems. Therefore, text preprocessing plays a significant role in improving the accuracy and efficiency of NLP applications.

In this experiment, three important text preprocessing techniques are performed on a Resume Dataset.

---

1. Tokenization

Tokenization is the process of splitting textual data into smaller meaningful units called tokens. Tokens can be words, characters, or sentences. Word tokenization is one of the most commonly used techniques in NLP because it converts a sentence into individual words that can be easily analyzed by machine learning algorithms.

Tokenization helps in understanding the structure of the text and forms the basis for various NLP tasks such as keyword extraction, information retrieval, text classification, and language modeling.

Example

Input:

```
The candidate has experience in Python, Machine Learning, and SQL.
```

Output:

```
['The', 'candidate', 'has', 'experience', 'in', 'Python', ',', 'Machine', 'Learning', ',', 'and', 'S
QL', '.']
```

---

2. Filtration

Filtration is the process of removing unnecessary or irrelevant information from textual data. Raw text often contains punctuation marks, numbers, extra spaces, special symbols, and unwanted characters that do not contribute significantly to text analysis.

Filtering the text helps in reducing noise and improving the overall quality of the dataset. Clean and filtered text allows NLP models to focus only on meaningful information present in resumes.

Filtration may include the removal of:

- Punctuation marks
- Numbers
- Special characters
- Extra white spaces
- Unwanted symbols

Example

Input:

```
Experience: Python, SQL, C++, and Machine Learning (3+ years).
```

Output:

```
Experience Python SQL C and Machine Learning years
```

---

3. Script Validation

Script validation is the process of verifying whether the textual data belongs to a particular writing system or language script. In this experiment, script validation is performed to ensure that the resume text contains only valid English alphabetic characters after filtration.

This technique is useful when resumes are collected from multiple sources where text may contain symbols or mixed language content. Script validation improves the reliability of the dataset and ensures consistency for NLP tasks.

Example

Input:

```
Skilled in Data Science and Artificial Intelligence.
```

Output:

```
Valid English Script
```

If the text contains unsupported characters or mixed scripts, the program identifies them accordingly.

Importance of Text Preprocessing

Text preprocessing provides several advantages in Natural Language Processing:

- Improves the quality and consistency of textual data.
- Removes unnecessary information from the dataset.
- Increases the performance of NLP models.
- Reduces computational complexity.
- Helps in preparing structured data for machine learning algorithms.
- Improves the accuracy of resume keyword extraction.
- Enables efficient text analysis and feature extraction.

Applications in Resume Keyword Extraction

The preprocessing techniques used in this experiment are widely applied in:

- Resume screening systems.
- Applicant Tracking Systems (ATS).
- Job recommendation systems.
- Skill extraction.
- Information retrieval systems.
- Recruitment automation.
- Text mining applications.

In [1]:
```
!pip install nltk spacy wordcloud
```

```
  Downloading preshed-3.0.13-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (779 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 779.7/779.7 KB 1.9 MB/s eta 0:00:00 MB/s eta 0:00:01:01
Collecting pydantic<3.0.0,>=2.0.0
  Downloading pydantic-2.13.4-py3-none-any.whl (472 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 472.3/472.3 KB 789.8 kB/s eta 0:00:00 kB/s eta 0:00:01:01
Requirement already satisfied: requests<3.0.0,>=2.13.0 in /home/computer/.local/lib/python3.10/site-packa
ges (from spacy) (2.34.2)
Requirement already satisfied: jinja2 in /home/computer/.local/lib/python3.10/site-packages (from spacy)
(3.1.6)
Collecting spacy-loggers<2.0.0,>=1.0.0
  Downloading spacy_loggers-1.0.5-py3-none-any.whl (22 kB)
Collecting cymem<2.1.0,>=2.0.2
  Downloading cymem-2.0.13-cp310-cp310-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (229 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 229.7/229.7 KB 662.0 kB/s eta 0:00:00[36m0:00:01m eta 0:00:01
Collecting typer<1.0.0,>=0.3.0
  Downloading typer-0.27.0-py3-none-any.whl (122 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 122.7/122.7 KB 5.8 MB/s eta 0:00:00
Requirement already satisfied: numpy>=1.19.0 in /home/computer/.local/lib/python3.10/site-packages (from
spacy) (2.2.6)
Collecting catalogue<2.1.0,>=2.0.6
```

In [2]:
```
import nltk

nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('omw-1.4')
```

```
[nltk_data] Downloading package punkt to /home/computer/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package stopwords to
[nltk_data]     /home/computer/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package wordnet to /home/computer/nltk_data...
[nltk_data] Downloading package omw-1.4 to /home/computer/nltk_data...
```

Out[2]:
```
True
```

In [3]:
```
import pandas as pd
import nltk
import string
import re

from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from collections import Counter
```

In [4]:
```
df = pd.read_csv("UpdatedResumeDataSet.csv")

df.head()
```

Out[4]:

	Category	Resume
0	Data Science	Skills * Programming Languages: Python (pandas...
1	Data Science	Education Details \r\nMay 2013 to May 2017 B.E...
2	Data Science	Areas of Interest Deep Learning, Control Syste...
3	Data Science	Skills â¯¢ R â¯¢ Python â¯¢ SAP HANA â¯¢ Table...
4	Data Science	Education Details \r\n MCA YMCAUST, Faridab...

In [7]:
```
# Select the first resume
resume = df.loc[0, "Resume"]

# Print only the first 200 characters
print(resume[:200] + "...")
```

```
Skills * Programming Languages: Python (pandas, numpy, scipy, scikit-learn, matplotlib), Sql, Java, JavaScr
ipt/JQuery. * Machine learning: Regression, SVM, NaÃ¯ve Bayes, KNN, Random Forest, Decision T...
```

In [8]:
```
# Convert resume text to lowercase
resume = resume.lower()

# Display only the first 200 characters
print(resume[:200] + "...")
```

```
skills * programming languages: python (pandas, numpy, scipy, scikit-learn, matplotlib), sql, java, javascr
ipt/jquery. * machine learning: regression, svm, naã¯ve bayes, knn, random forest, decision t...
```

In [10]:
```
from nltk.tokenize import word_tokenize

tokens = word_tokenize(resume)

# Display only the first 20 tokens
print(tokens[:20])
```

```
['skills', '*', 'programming', 'languages', ':', 'python', '(', 'pandas', ',', 'numpy', ',', 'scipy', ',',
'scikit-learn', ',', 'matplotlib', ')', ',', 'sql', ',']
```

In [11]:
```
import string

tokens = [word for word in tokens if word not in string.punctuation]

# Display only the first 20 filtered tokens
print(tokens[:20])
```

```
['skills', 'programming', 'languages', 'python', 'pandas', 'numpy', 'scipy', 'scikit-learn', 'matplotlib',
'sql', 'java', 'javascript/jquery', 'machine', 'learning', 'regression', 'svm', 'naã¯ve', 'bayes', 'knn',
'random']
```

In [12]:
```
from nltk.corpus import stopwords

stop_words = set(stopwords.words("english"))

filtered = [word for word in tokens if word not in stop_words]

# Display only the first 20 words
print(filtered[:20])
```

```
['skills', 'programming', 'languages', 'python', 'pandas', 'numpy', 'scipy', 'scikit-learn', 'matplotlib',
'sql', 'java', 'javascript/jquery', 'machine', 'learning', 'regression', 'svm', 'naã¯ve', 'bayes', 'knn',
'random']
```

In [14]:
```
import re

if re.match(r'^[A-Za-z0-9\s.,()\-+/]*$', resume):
    print("Script Validation: English Script")
else:
    print("Script Validation: Other Language Found")
```

```
Script Validation: Other Language Found
```

In [15]:
```
from collections import Counter

word_count = Counter(lemmatized)

# Show only the top 10 keywords
print(word_count.most_common(10))
```

```
[('data', 11), ('analysis', 8), ('review', 8), ('tool', 8), ('information', 8), ('technology', 6), ('analyt
ics', 6), ('python', 5), ('word', 5), ('topic', 5)]
```

In [16]:
```
top_keywords = word_count.most_common(10)

for word, count in top_keywords:
    print(f"{word} : {count}")
```

```
data : 11
analysis : 8
review : 8
tool : 8
information : 8
technology : 6
analytics : 6
python : 5
word : 5
topic : 5
```